Encapsulation in Python

Encapsulation means:

- Wrapping **data (variables)** and **methods (functions)** into a single unit (class).
- Controlling **how data is accessed or modified**.
- Protecting sensitive data by hiding internal details.

In Python, encapsulation is shown using **public, protected, and private variables**.

---

1) Public Variable (default)

Public variables are accessible from **anywhere**: inside the class, outside the class, or by objects.

```
class Demo:
    def __init__(self):
        self.name = "Public Variable"   # public


obj = Demo()
print(obj.name)   # Accessible
```

Explanation:

- self.name is **public**.
- We can directly use obj.name without any restriction.

---

2) Protected Variable (_var)

Protected variables are defined with a **single underscore (_)**.

- They are still accessible outside the class.
- But **by convention**, they should be used **only inside the class or subclasses** (not by normal code).

```
class Demo:
    def __init__(self):
        self._name = "Protected Variable"   # protected


obj = Demo()
print(obj._name)   # ⚠ Accessible but not recommended
```

Explanation:

- self._name is **protected**.

- You *can* access it using obj._name, but it signals: **"Don't touch this directly unless you are inside a subclass."**

---

3) Private Variable (__var)

Private variables are declared with **double underscores (__)**.

- They are **not directly accessible** outside the class.
- Python internally changes their name (called **name mangling**) to prevent accidental access.

```
class Demo:
    def __init__(self):
        self.__name = "Private Variable"   # private


obj = Demo()
# print(obj.__name)   # Error
print(obj._Demo__name)  # Access using name-mangling
```

Explanation:

- self.__name is private.
- If you try obj.__name, Python throws an **AttributeError**.
- But Python stores it as _Demo__name, so you *can* still access it using that name (not recommended).

Final Summary

- **Public (name)** → free access anywhere.
- **Protected (_name)** → accessible, but should only be used inside the class or subclasses (just a warning convention).
- **Private (__name)** → not directly accessible, only through class methods or name-mangling.
